ProjetLocatif — Rapport de posture de sécurité (périmètre : compte propre)

Intervenant : Nicolas BRANDEJSKY (compte 8570) · **Date :** 2026-11-05 **Périmètre :** `app.projetlocatif.fr` — tests limités au **compte propre**, en **lecture seule**, **sans aucun accès à des données de tiers** et **sans modification**. **Nature :** vérification de configuration (cookies, en-têtes, authentification, CSRF). Ce rapport **ne couvre pas** la question de l'étanchéité entre comptes (IDOR) — voir §4.

> *Document confidentiel, destiné à l'éditeur de la plateforme (divulgation responsable).*

---

1. Ce qui fonctionne bien ✅

Point vérifié	Résultat
Cookie de session `HttpOnly`	✅ Oui (non lisible par JavaScript)
Protection CSRF / jeton sur écriture	✅ POST refusé **401** sans l'en-tête `X-XSRF-TOKEN`, **200** avec
Accès API sans authentification	✅ Refusé (redirection vers login)
`X-Frame-Options: SAMEORIGIN` (anti-clickjacking)	✅ Présent
`X-Content-Type-Options: nosniff`	✅ Présent

2. Points à corriger (configuration)

#	Gravité	Constat	Recommandation
F1	Moyenne	Cookies **sans `Secure` ni `SameSite`** (`projetlocatif_session`, `laravel_token`, `XSRF-TOKEN`)	Ajouter `Secure` (cookies envoyés uniquement en HTTPS) et `SameSite=Lax` (défense CSRF en profondeur)
F2	Moyenne	**HSTS absent** (`Strict-Transport-Security`)	Ajouter l'en-tête HSTS pour forcer HTTPS
F3	Faible	**CSP absente** (`Content-`	Définir une CSP

#	Gravité	Constat	Recommandation
	blee → Moyenne	`Security-Policy`)	(défense anti-XSS en profondeur)
F4	Faible	`laravel_token` / `XSRF-TOKEN` avec **durée ~1 an** (`Max-Age` → 2027)	Réduire la durée de vie des cookies/jetons
F5	Faible	`Referrer-Policy` et `Permissions-Policy` absents	Les ajouter (bonnes pratiques)
F6	Info	Le **modèle de permissions** (`authorisations`) et la **table complète des routes** sont renvoyés au client	Limiter l’exposition (facilite la reconnaissanc

#	Gravité	Constat	Recommandation
F7	Info	Appel API non authentifié → **302 vers login** au lieu d'un `401` JSON	Renvoyer `401` sur les routes API (confort d'intégration)

3. Détails techniques (preuves, sur compte propre)

- **En-têtes réponse** (`/prospects`, authentifié) : présents → `X-Frame-Options: SAMEORIGIN`, `X-Content-Type-Options: nosniff`, `X-XSS-Protection: 1; mode=block` ; absents → HSTS, CSP, Referrer-Policy, Permissions-Policy.
- **Cookies** : `projetlocatif_session` → `HttpOnly`, **pas** de `Secure`, **pas** de `SameSite` ; `laravel_token` → `HttpOnly`, `Max-Age` ~1 an ; `XSRF-TOKEN` → lisible par JS (attendu pour le double-submit), `Max-Age` ~1 an.
- **CSRF** : `POST /api/prospects/search` → **401** sans `X-XSRF-TOKEN`, **200** avec. La protection par jeton est donc **effective** sur les requêtes d'écriture (ce qui compense largement l'absence de `SameSite`).
- **Non authentifié** : `GET /api/prospects/options` sans cookie → **302** (login). Aucune donnée renvoyée.

4. Ce que ce rapport NE dit PAS (et comment le compléter)

La question centrale de Robin — *« un conseiller peut-il accéder à des données hors de son périmètre ? »* (**IDOR / autorisation objet par objet**) — **n'est pas tranchée ici**, parce qu'y répondre impose de tenter d'accéder à des données d'un **autre** compte, ce qui ne doit pas se faire sur la production avec des données clients réelles.

Pour y répondre proprement et sans risque : l'éditeur fournit un **environnement de recette** + **deux comptes de test A et B** (qui ne doivent pas se voir). On teste alors

l’étanchéité objet par objet entre A et B (lecture puis écriture), on prouve la présence/absence d’IDOR sur un **cas témoin**, puis on arrête. Zéro donnée réelle exposée.

5. Suite recommandée

1. Corriger F1 → F5 (configuration rapide, fort rapport bénéfice/effort).
2. Mettre en place la **recette + comptes A/B** pour le test d’étanchéité (§4).
3. Côté serveur : s’assurer que chaque route `show/update/destroy` vérifie l’**appartenance de l’objet** au périmètre de l’utilisateur (Policies Laravel), pas seulement le rôle.